

Introduction to Algorithm

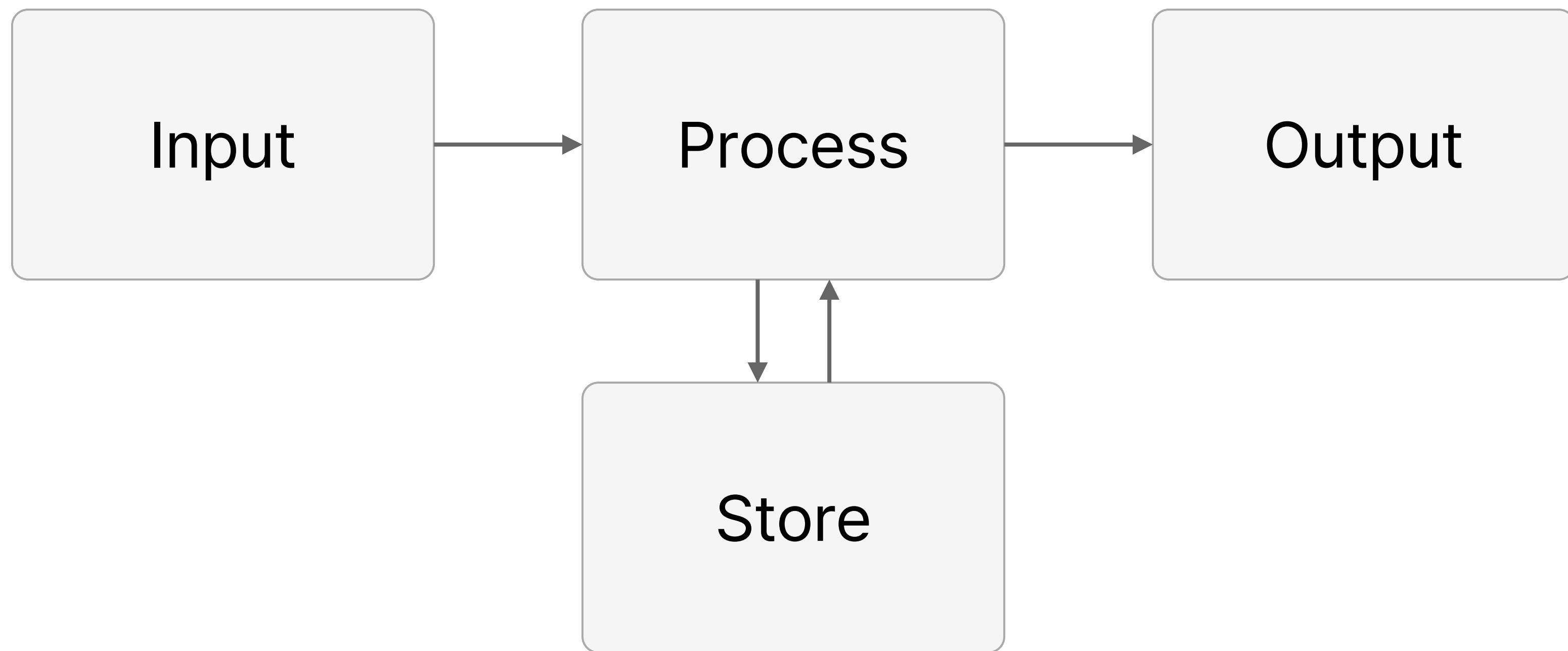
기초 알고리즘

알고리즘의 개념

알고리즘과 자료구조

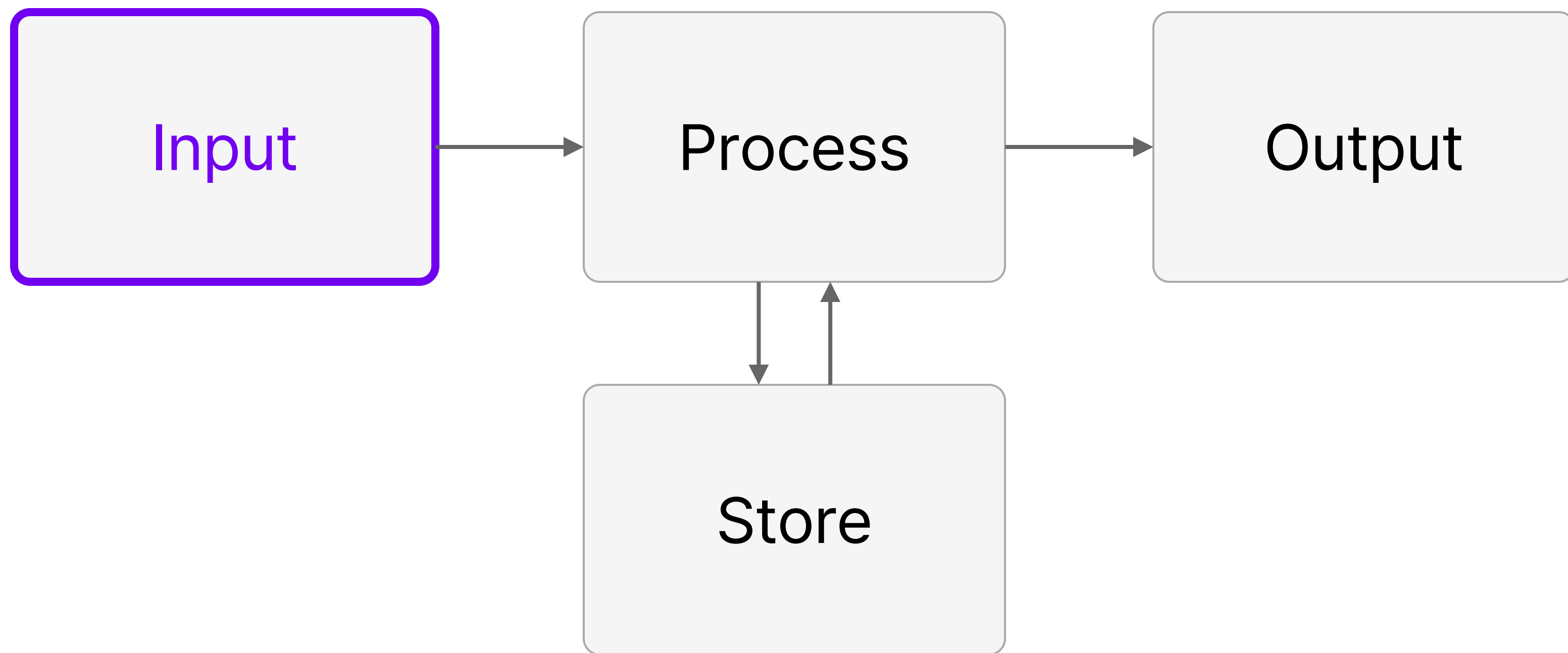


프로그램의 기본 구성



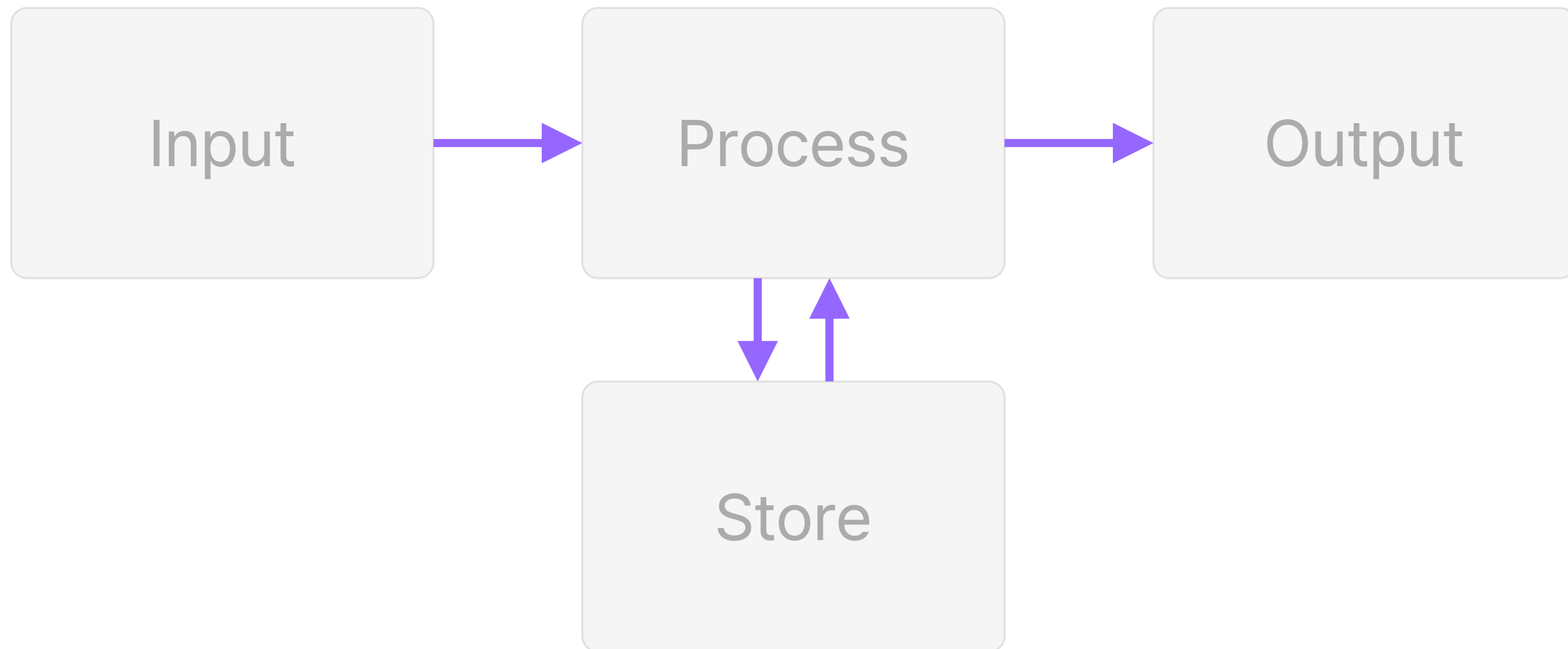


프로그램의 기본 구성



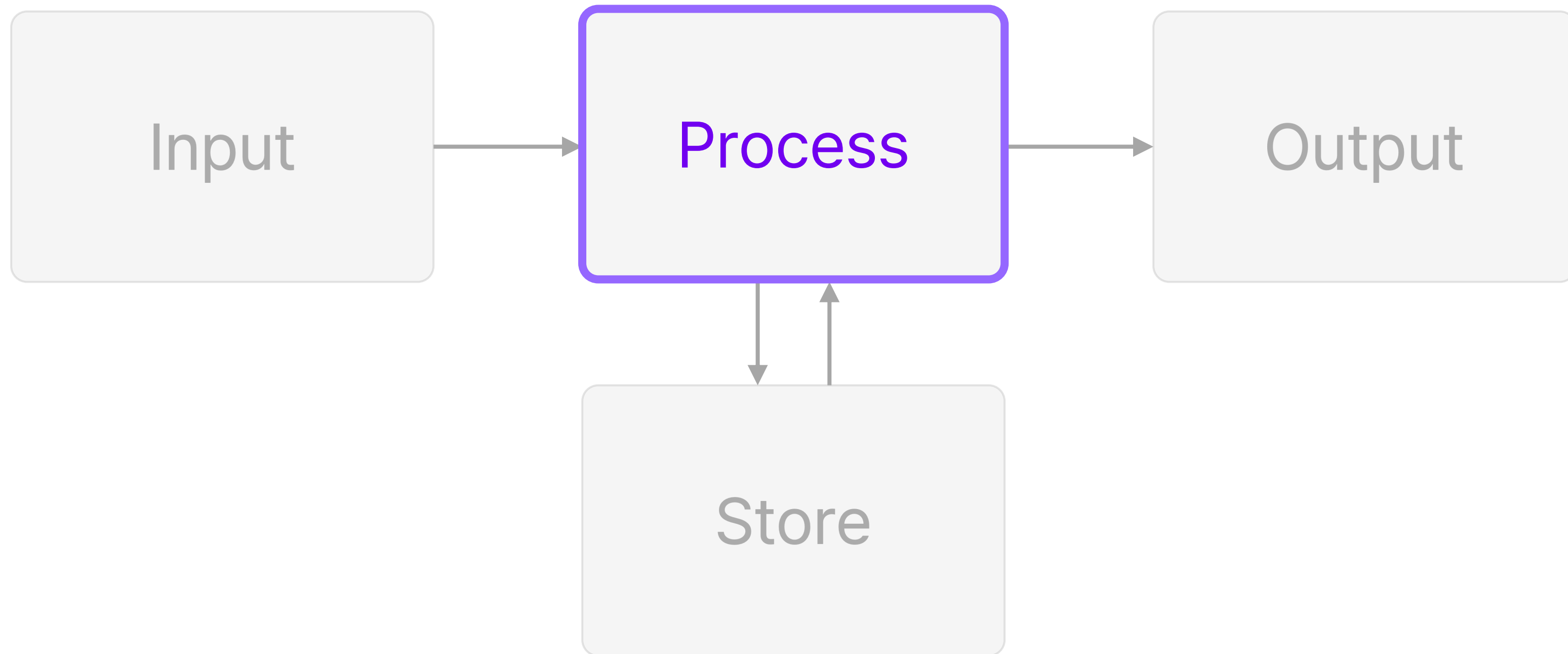


데이터의 흐름과 저장 - 자료구조





시간과 공간 효율성 - 알고리즘





자료구조의 종류





때와 장소에 맞는 자료구조를 사용해야 함에 주의

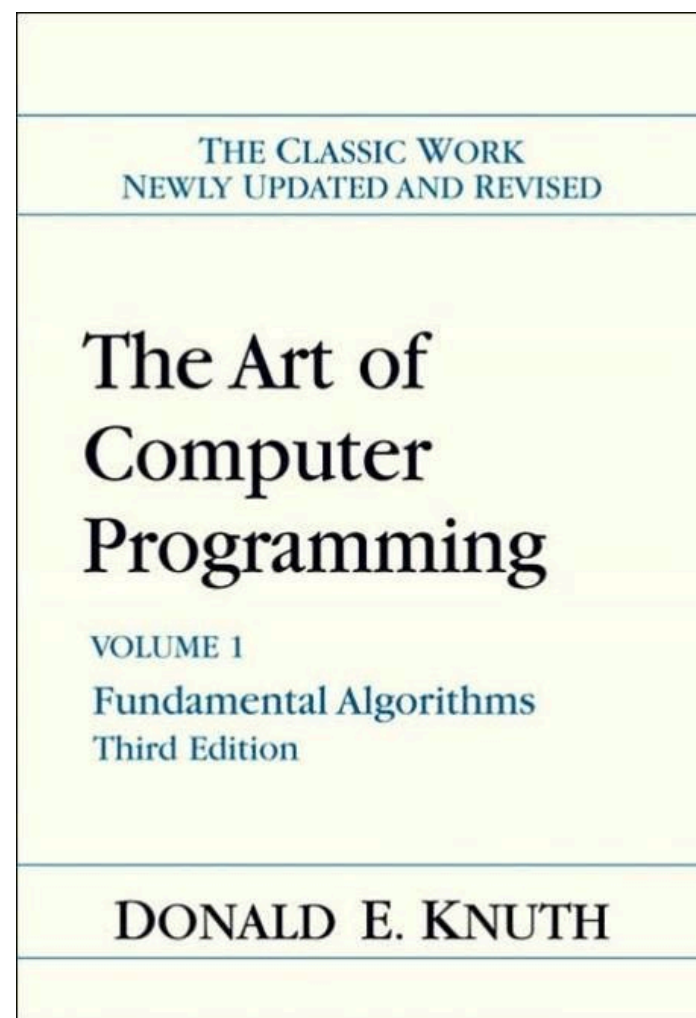


좋은 알고리즘의 조건



Donald Knuth

컴퓨터 과학의 아버지로 불리는 인물



Input (입력)

Output (출력)

Definiteness (명확성)

Finiteness (유한성)

Effectiveness (효과성)



좋은 알고리즘의 조건

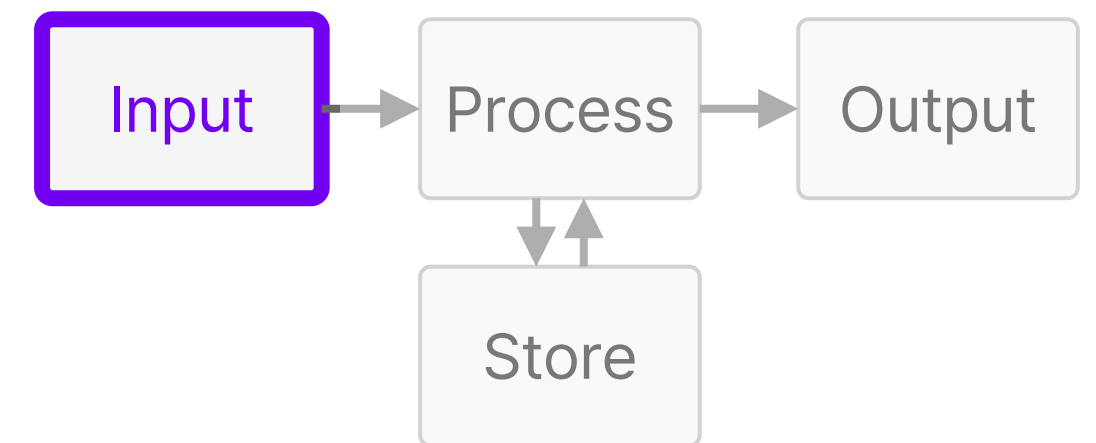
Input (입력)

Output (출력)

Definiteness (명확성)

Finiteness (유한성)

Effectiveness (효과성)



알고리즘은 **0개 이상의 외부 입력**을 받아야 한다

- 입력이 없을 수도 있음 (예: 난수 생성, 현재 시간 출력)
- 입력이 있다면 그 범위와 형식이 명확히 정의되어야 함
- "어떤 데이터가 들어올 수 있는가"가 명시되어야 함

예시: 정렬 알고리즘 → 정수 배열, 문자열 리스트 등 입력 타입이 정의됨



좋은 알고리즘의 조건

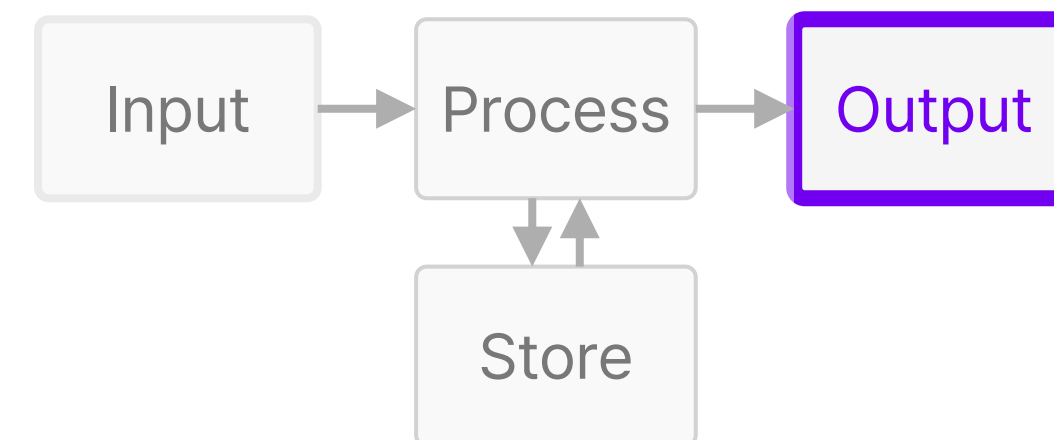
Input (입력)

Output (출력)

Definiteness (명확성)

Finiteness (유한성)

Effectiveness (효과성)



알고리즘은 **1개 이상의 결과**를 반드시 출력해야 한다

- 입력과 달리 출력은 반드시 존재해야 함
- 출력이 없으면 알고리즘의 존재 의미가 없음
- 출력은 입력과 명확한 관계를 가져야 함

예시: 탐색 알고리즘 → 찾으면 위치 반환, 못 찾으면 -1 또는 False 반환



좋은 알고리즘의 조건

Input (입력)

Output (출력)

Definiteness (명확성)

Finiteness (유한성)

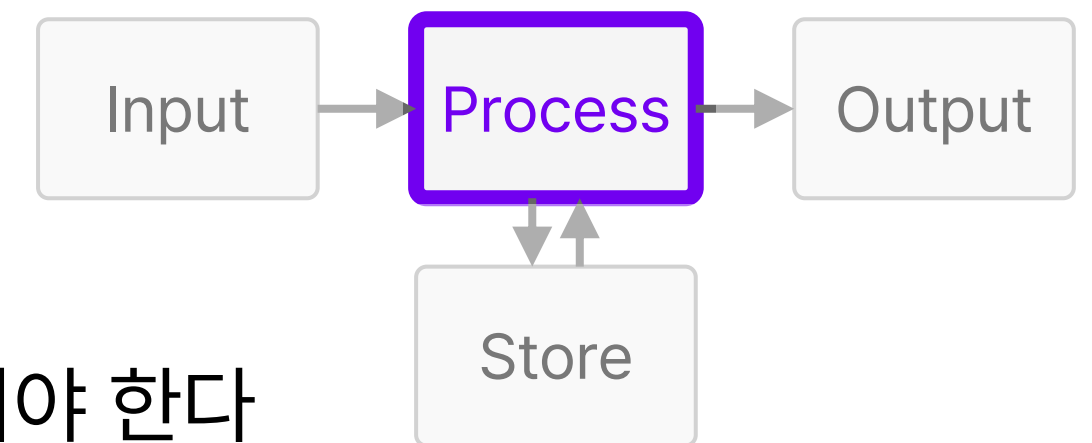
Effectiveness (효과성)

각 단계가 모호하지 않고 **명확하게 정의**되어야 한다

- 누가 실행하든 동일하게 해석되어야 함
- "적당히", "대충" 같은 모호한 표현 불가
- 컴퓨터가 해석할 수 있을 정도로 구체적이어야 함

(X) "큰 수를 앞으로 보낸다" (큰 수의 기준이 뭔지 불명확)

(O) "i번째 원소가 i+1번째 원소보다 크면 두 원소의 위치를 교환한다"





좋은 알고리즘의 조건

Input (입력)

Output (출력)

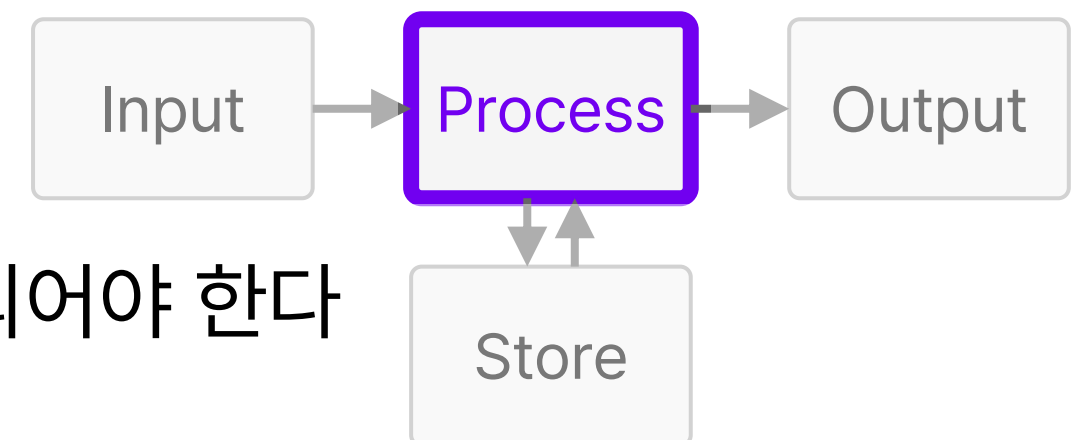
Definiteness (명확성)

Finiteness (유한성)

Effectiveness (효과성)

알고리즘은 유한한 단계 후에 반드시 **종료**되어야 한다

- 무한 루프에 빠지면 알고리즘이 아님
- 모든 경로에서 종료 조건에 도달해야 함
- "언젠가는 끝난다"가 보장되어야 함



나쁜 예시

```
while True:
    print("무한반복")
    # 종료 조건 없음
```

좋은 예시

```
for i in range(n):
    # n번 후 반드시 종료
    process(i)
```



좋은 알고리즘의 조건

Input (입력)

Output (출력)

Definiteness (명확성)

Finiteness (유한성)

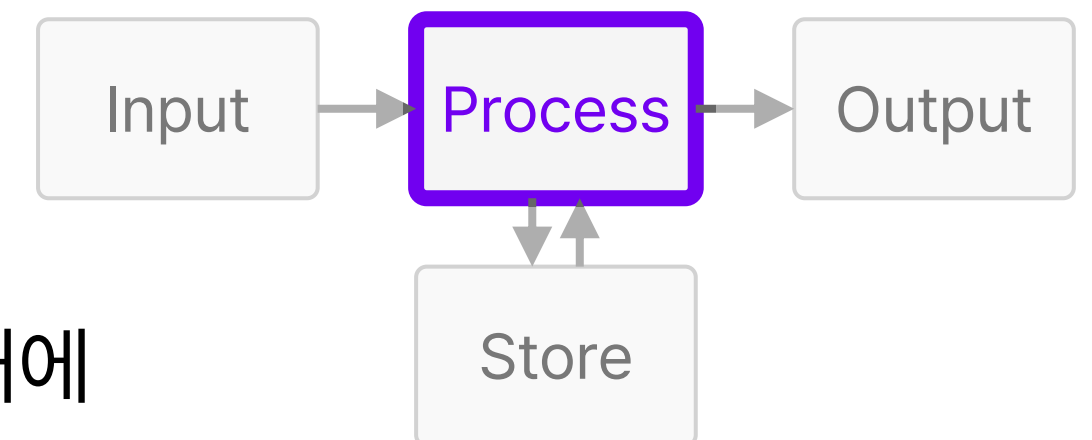
Effectiveness (효과성)

모든 연산이 종이와 연필로 유한한 시간 내에 수행 가능할 정도로 기본적인어야 한다.

- 이론적으로만 가능한 게 아니라 실제로 계산 가능해야 함
- 기본적인 연산들로 구성되어야 함

(X) " π 의 모든 자릿수를 구하라" (무한하므로 실행 불가)

(O) " π 를 소수점 10자리까지 구하라"



Python 핵심 자료구조

자료구조	특징	관련 알고리즘
배열	인덱스 접근 $O(1)$	정렬, 이진탐색
스택	LIFO	DFS, 백트래킹
큐	FIFO	BFS
힙	최대/최소값 빠른 접근	Dijkstra, Prim
그래프	노드 + 간선	DFS, BFS, 최단경로, MST
서로소 집합 (Union-Find)	집합 병합/조회	Kruskal, 사이클 탐지



리스트 (List) : 변수의 나열

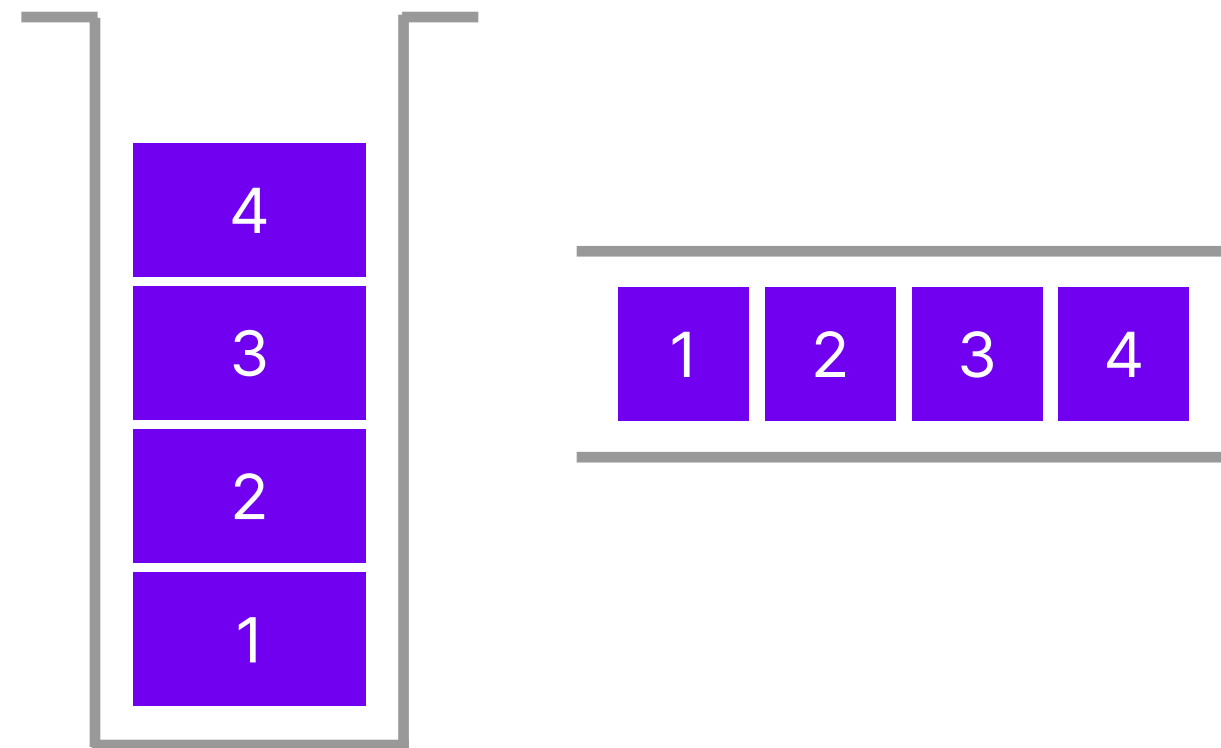
주소 id	32	64	96	128	160	192	224	256
index	0	1	2	3	4	5	6	7
myList	3	4	2	5	1	2	6	3

장점 : i 번째 원소를 바로 알 수 있다 (myList[i])

단점 : 원소의 추가/삭제가 까다롭다

대표적인 자료 구조의 예시

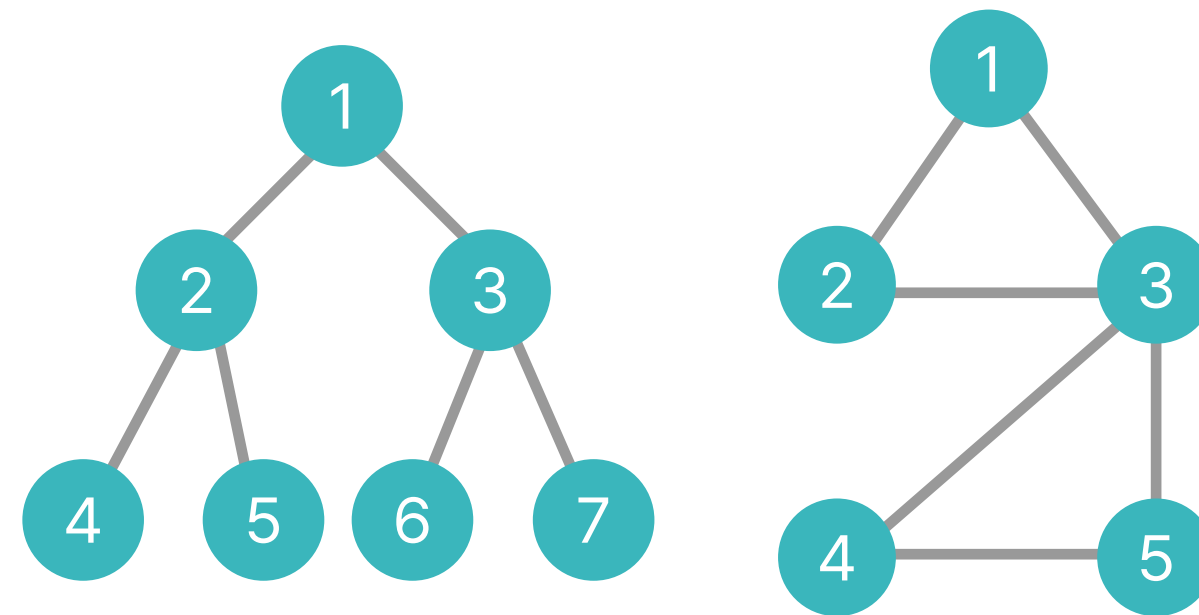
선형 구조



스택 (Stack)

큐 (Queue)

비선형 구조

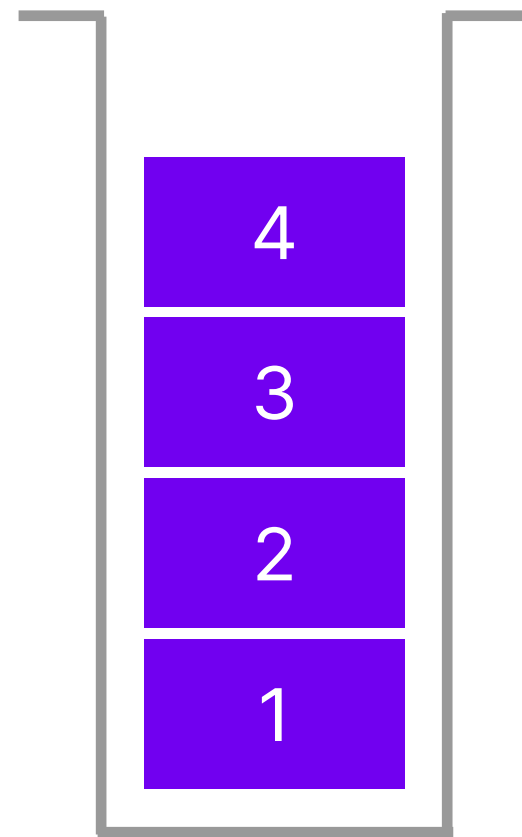


트리 (Tree)

그래프 (Graph)

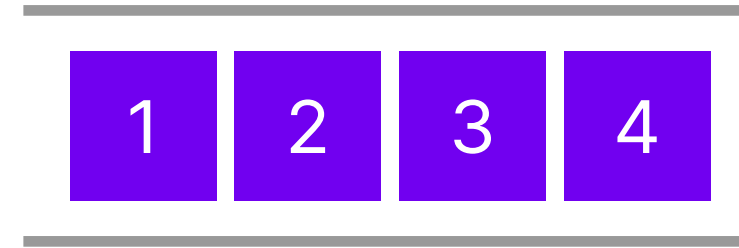
대표적인 자료 구조의 예시

push
pop



스택 (Stack)

Last In First Out



큐 (Queue)

First In First Out

put
get
qsize



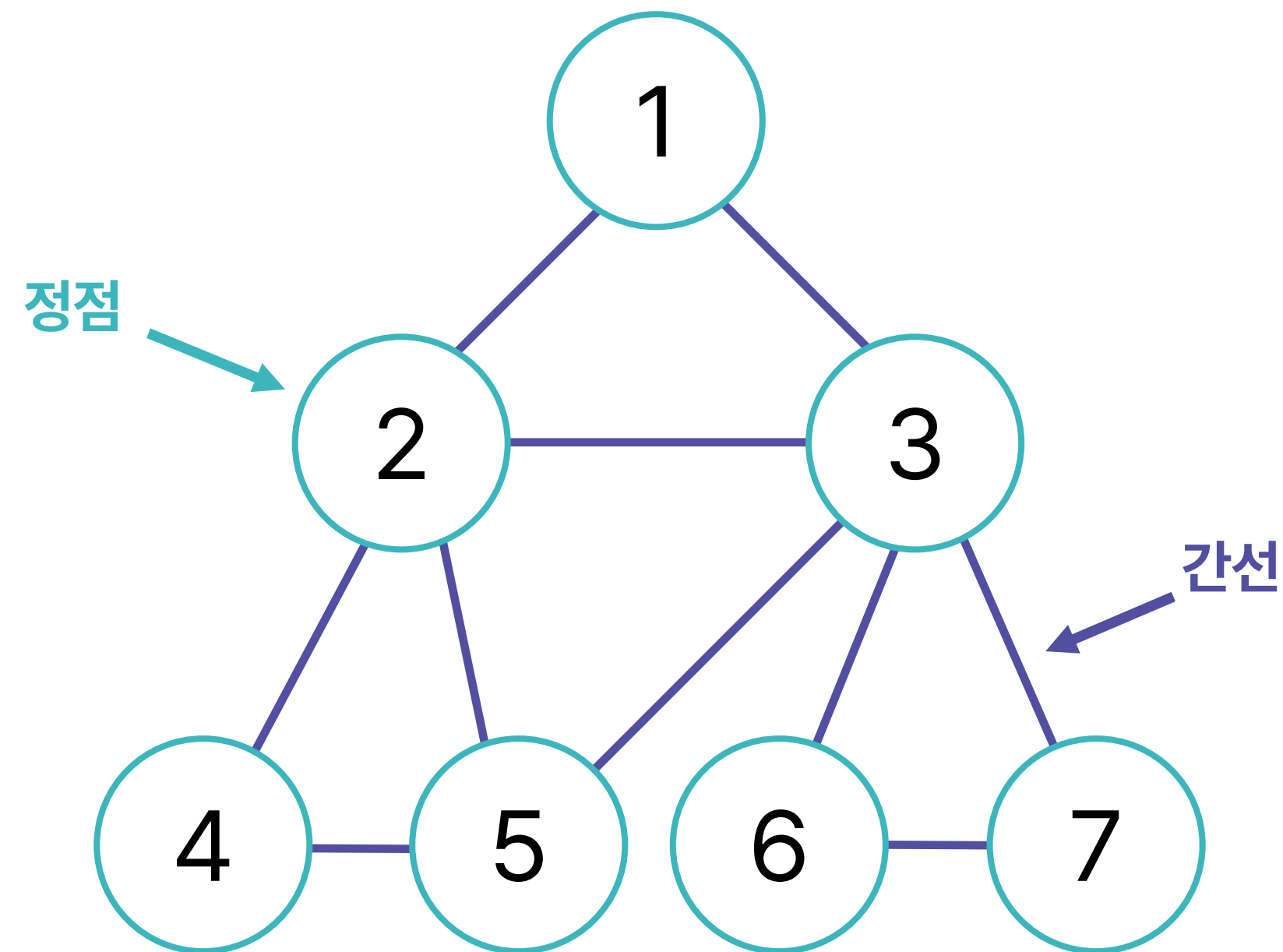
그래프

정점(vertex)과 간선(edge)으로 이루어져 있는 자료구조

정점(vertex): 자료, 상태 등 뭔가를 담고 있음

간선(edge): 정점 간의 관계를 나타냄

(정점은 '노드'라고도 표현한다.)

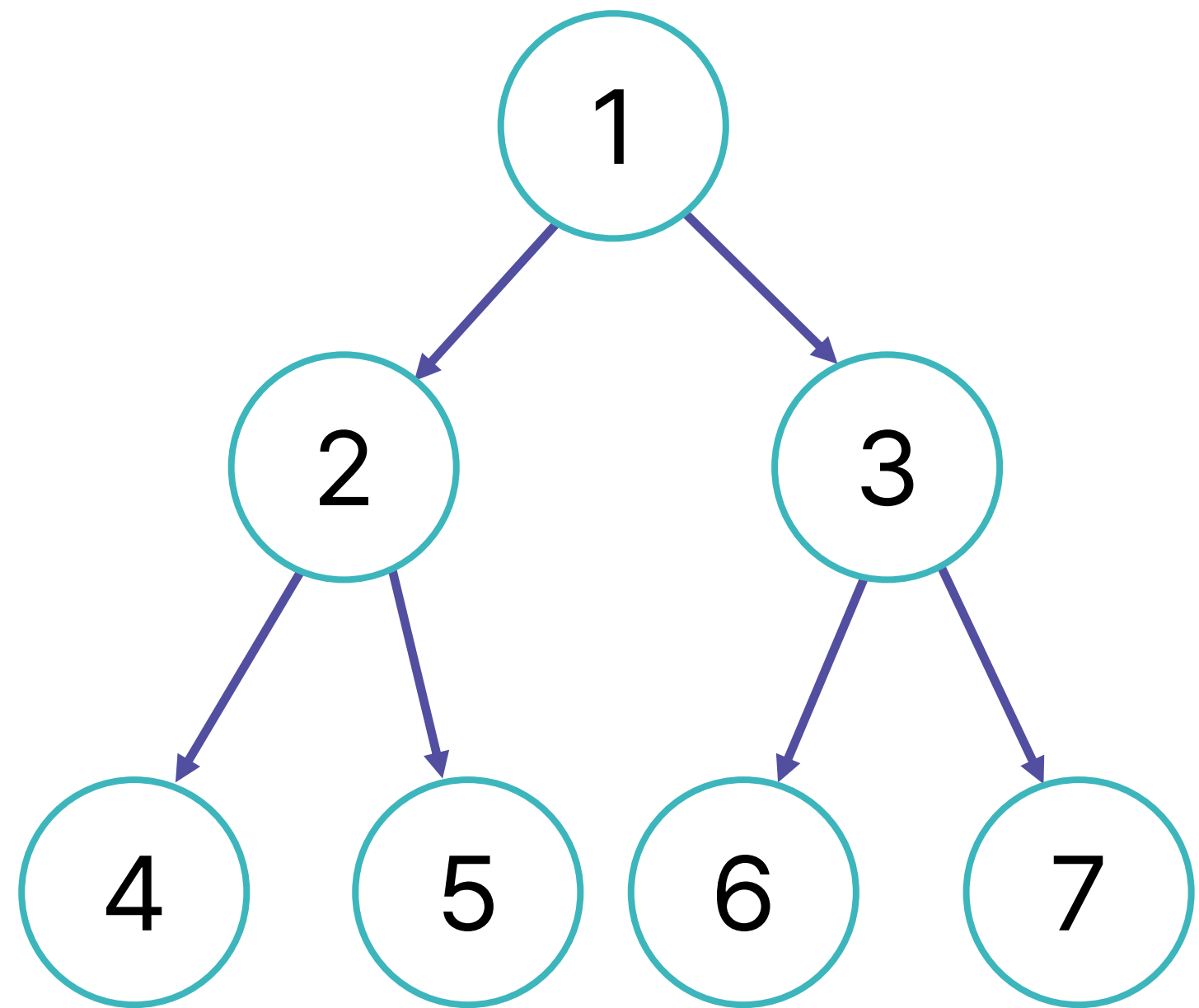


트리

그래프 중 아래의 특별한 성질을 갖는 그래프를 트리라고 한다.

트리의 여러 성질은 다음과 같다.

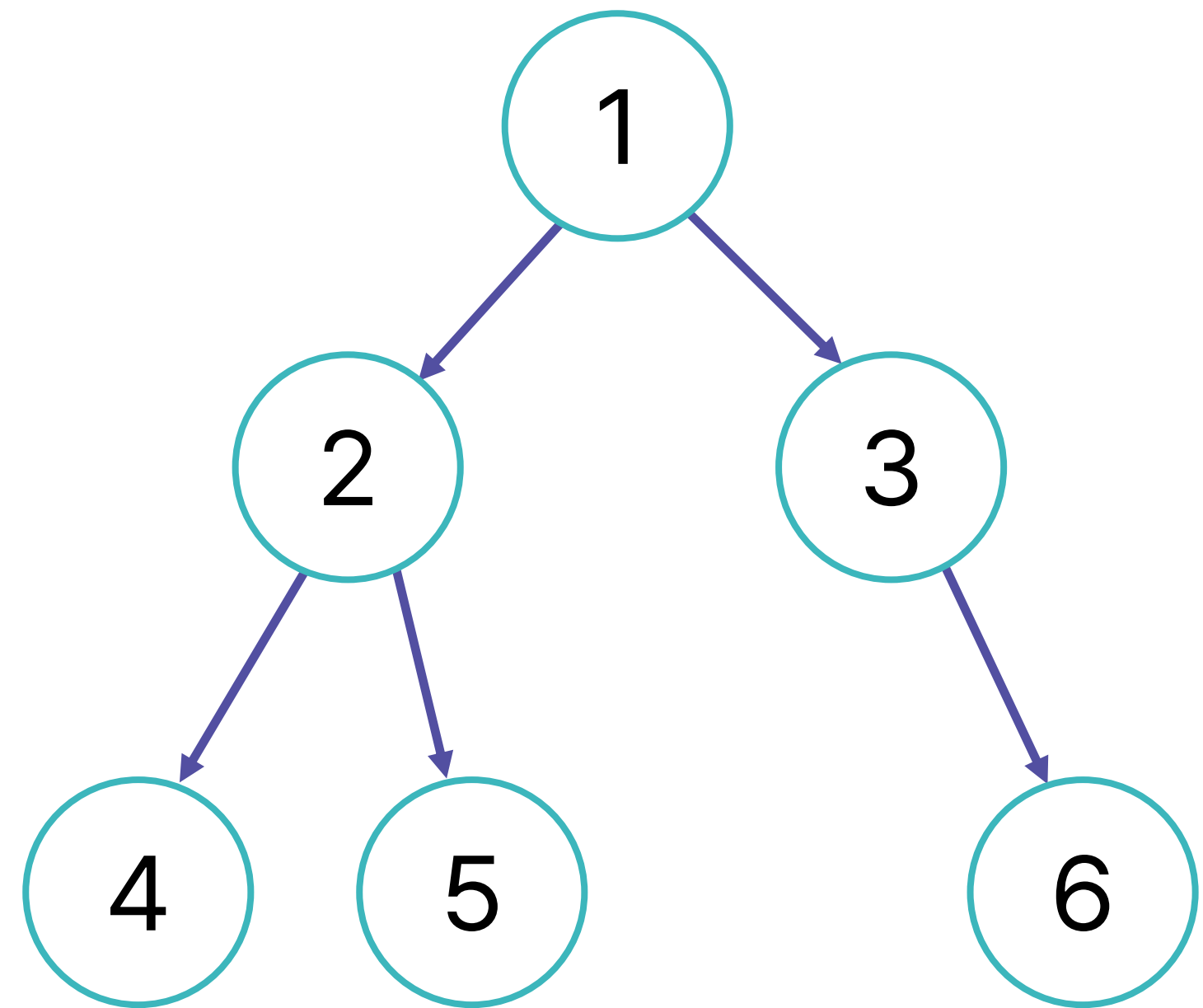
- 트리의 간선들은 모두 방향성을 갖는다.
- 어떤 정점을 가리키는 정점의 개수는 최대 1개이다.
- 어떤 정점에서 다른 정점으로 이동할 수 있는 경로는 1개다.
- 트리는 사이클을 갖지 않는다.



이진 트리

각 정점들이 자식 노드를 **최대 2개**까지만 갖는 트리를 **이진 트리**라고 한다.

이진 탐색 트리 등 유용하게 활용되는 트리 중에는 대부분 **이진 트리**를 응용한 것이 많다.

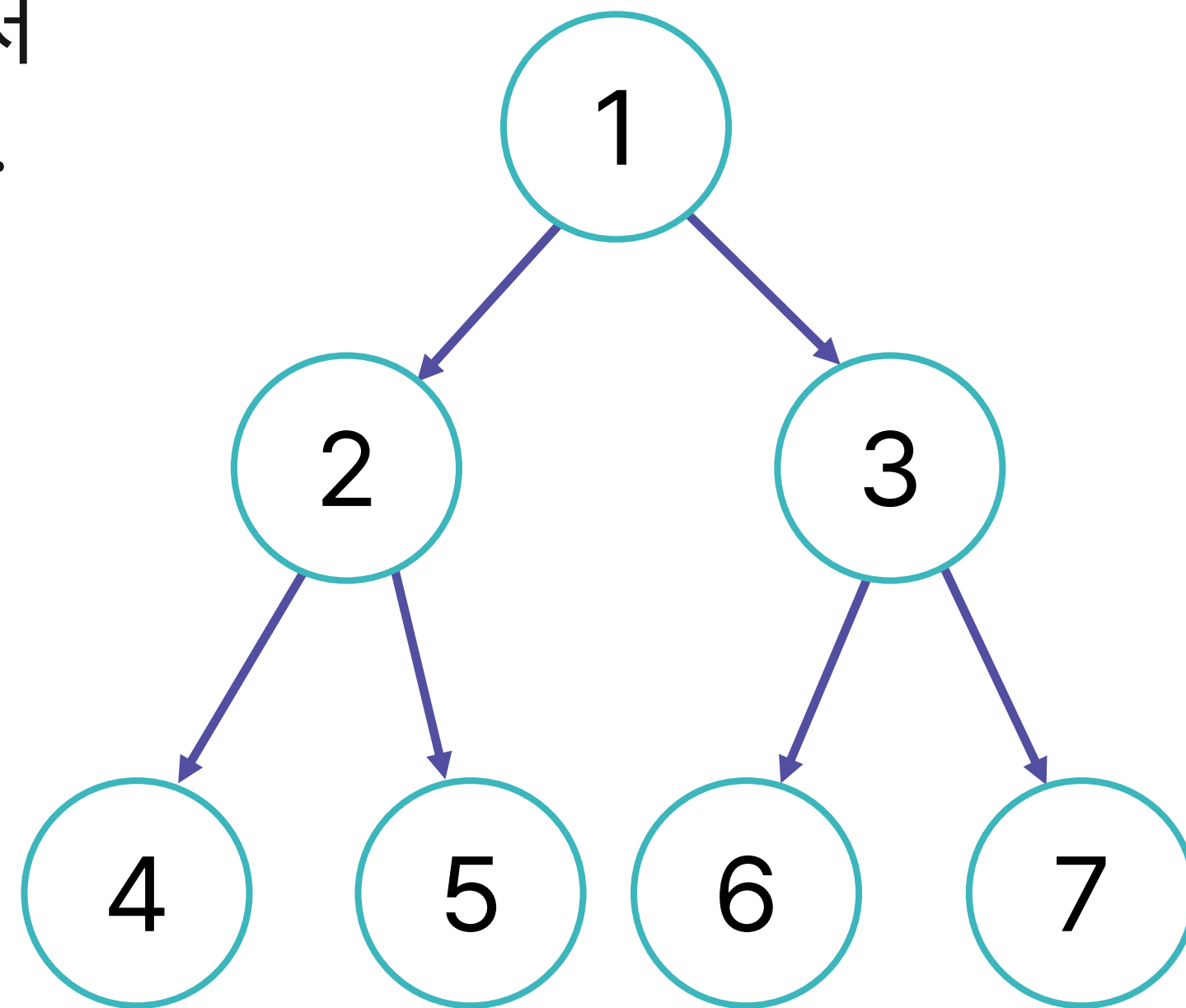




포화 이진 트리

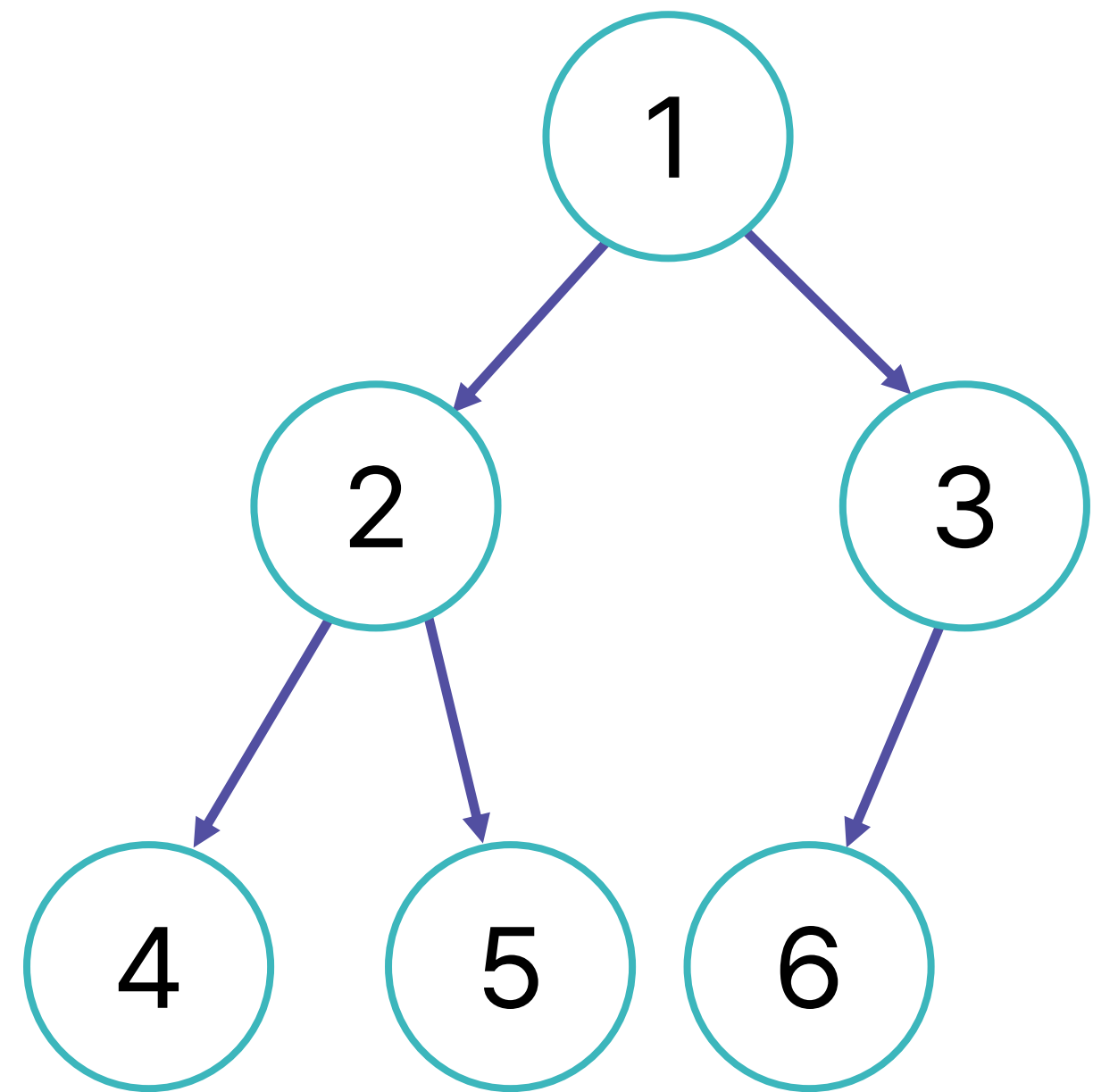
리프 노드를 제외한 모든 정점이 항상 자식을 2개씩 갖고 있으면서 모든 리프 노드의 깊이가 동일한 트리를 포화 이진 트리라고 한다.

포화 이진 트리의 높이를 h 라고 할 때, 정점의 개수는 항상 $2^h - 1$ 개이다.



완전 이진 트리

마지막 깊이를 제외하고 모든 정점이 **완전히** 채워져 있으며,
마지막 깊이의 정점들은 가능한 한 **왼쪽**에 있는 트리를
완전 이진 트리라고 한다.

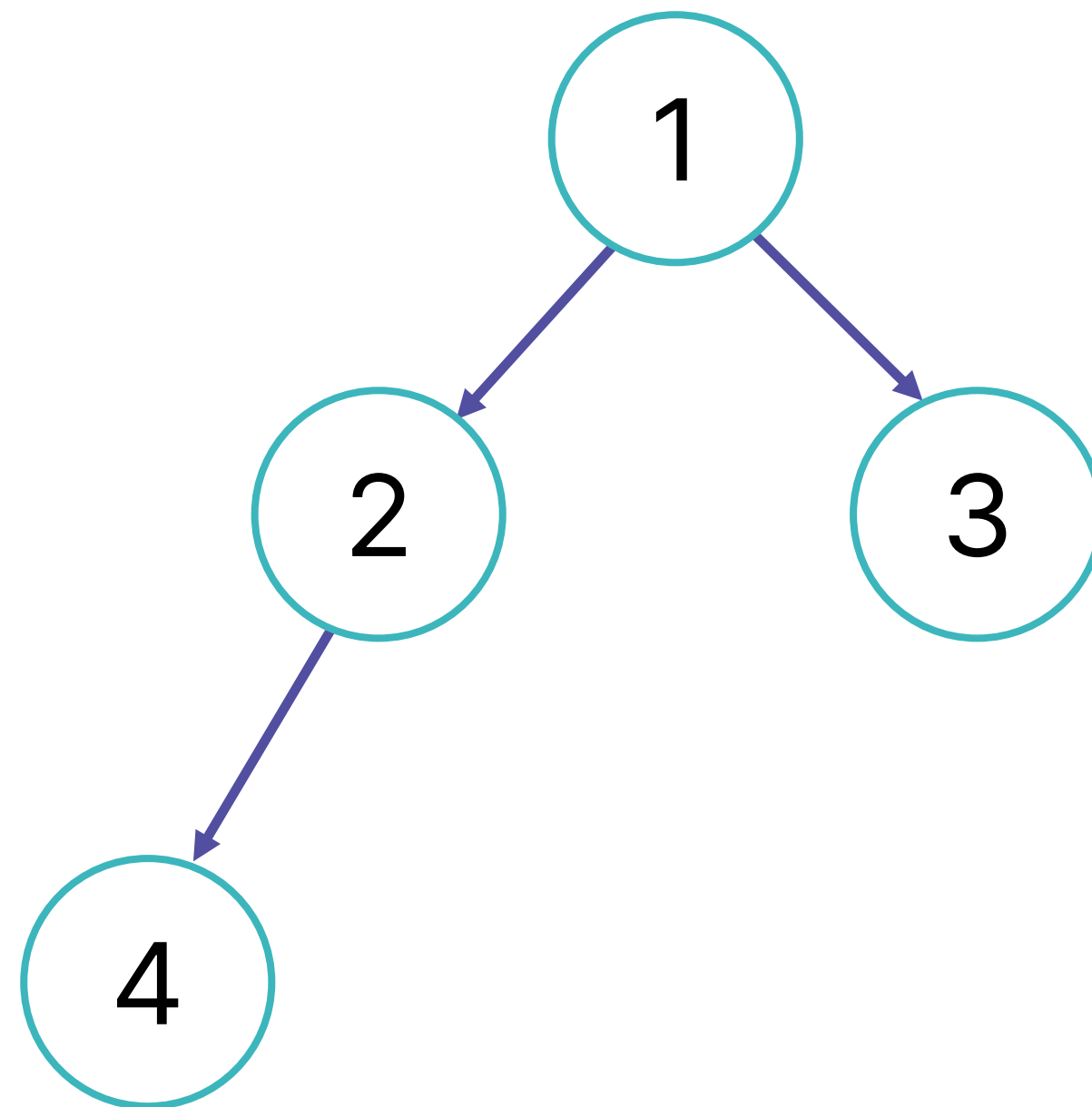




완전 이진 트리

또는 **포화 이진 트리**에서 마지막 깊이의 정점이 **오른쪽**에서부터 **일부 제거**된 트리라고 볼 수 있다.

완전 이진 트리의 높이가 h 일 때
정점의 개수는 2^{h-1} 개 이상 $2^h - 1$ 개 이하이다.



힙

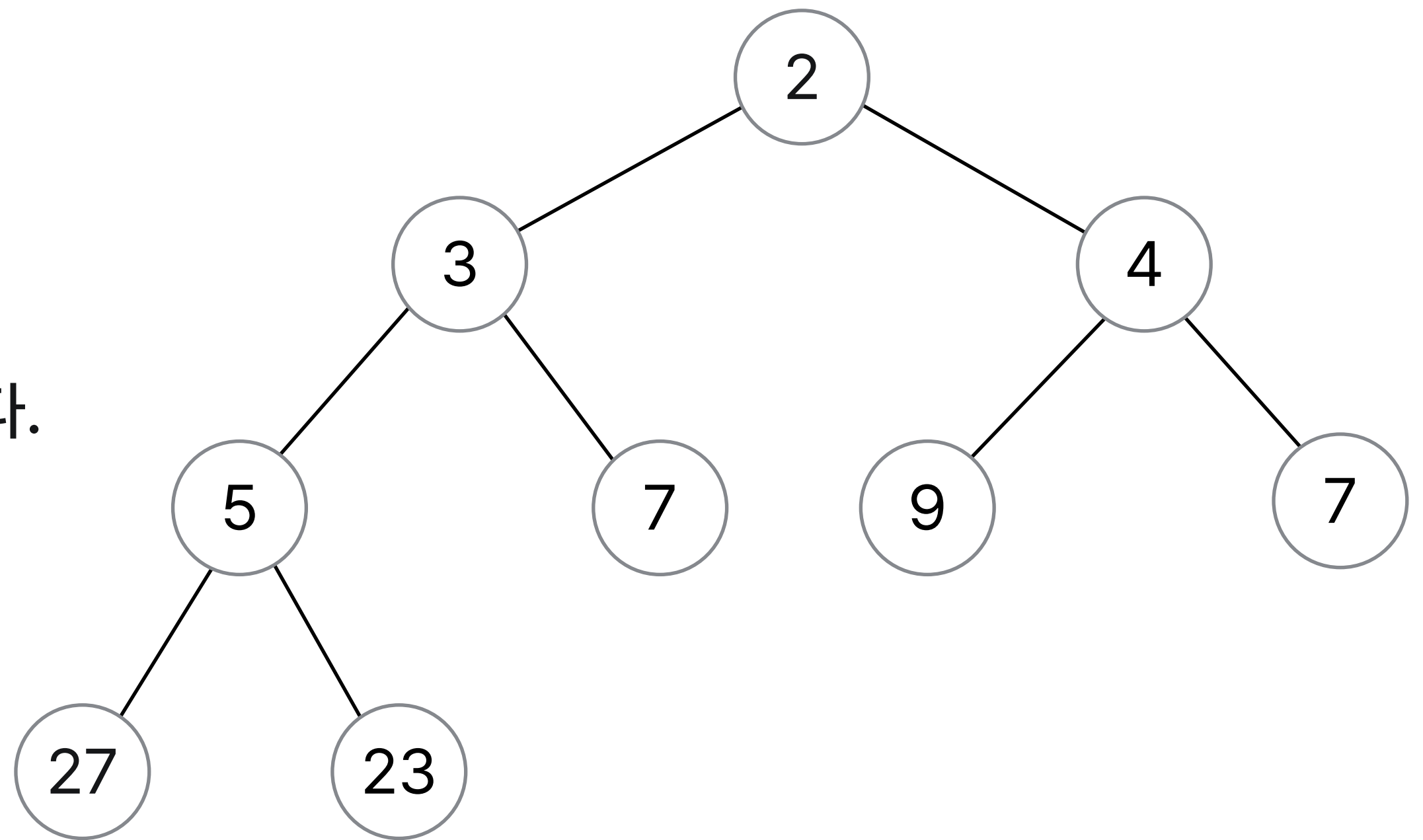
최솟값 또는 최댓값을 빠르게
찾기 위해 고안된 완전 이진 트리

최대 힙(Max Heap)

부모 노드는 항상 자식 노드보다 큰 값을 갖고 있다.

최소 힙(Min Heap)

부모 노드는 항상 자식 노드보다 작은 값을
갖고 있다.



시간 복잡도와 공간 복잡도



전화번호를 찾는 방법 → 알고리즘

- 문제를 해결하는 방법(시간과 공간 효율성)

시간과 공간 효율성이란?



시간 복잡도 및 공간 복잡도

- 복잡도의 개념
 - 입력 크기(N)에 따라서 얼마나 커지는지(실행 시간 or 메모리 공간 사용량)
- 시간 복잡도 → 문제를 해결하는 데 걸리는 시간과 입력 함수의 관계를 가리킴
- 공간 복잡도 → 문제를 해결하는 데 필요한 메모리 공간 사용량과 입력 함수의 관계



효율적인 알고리즘에 대한 고민

- 요리사가 같은 시간 안에 더 많은 요리를 만들기 위해서는?
- 시간 복잡도(Time Complexity)는 문제를 해결하는 데 걸리는 시간과 입력의 함수 관계를 가리킴





단순한 연산 횟수 계산

```
sum = 0;  
for i in [1, 2, 3, 4]:  
    sum += i;
```

시간 복잡도 =



단순한 연산 횟수 계산

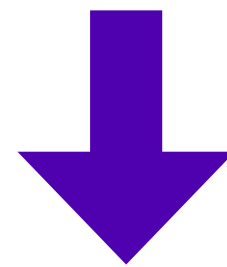
```
randomNumber = 0
nums = [1, 2, 3, 4]
for i in range(len(nums)):
    for j in range(len(nums)):
        randomNumber += nums[i] * nums[j]
```

시간 복잡도 =



입력 변수의 크기가 N이라면?

입력 변수의 크기 = N



코드의 시간 복잡도 = $f(N)$



시간 복잡도 = 1

```
def doNothing(nums):  
    return nums
```




시간 복잡도 = $N + 2$

```
def doSomething(nums):  
    sum = 0  
    for num in nums:  
        sum += num  
    return sum
```




$$\text{시간 복잡도} = 2 \times N^2 + 2$$

```
def doManythings(nums):  
    allPairs = []  
    for i in range(len(nums)):  
        for j in range(len(nums)):  
            if nums[i] < nums[j]:  
                allPairs.append((nums[i], nums[j]))  
            else:  
                allPairs.append((nums[i], nums[j]))  
    return allPairs
```



복잡도의 중요성?

구분	코드 1	코드 2
시간 복잡도	1,000N	2N ²
Big-O	O(N)	O(N ²)

N	1	10	100	1,000	10,000	...
코드 1	1,000	10,000	100,000	1,000,000	10,000,000	...
코드 2	2	200	20,000	2,000,000	200,000,000	...



여러 가지 복잡도 표기 방법

- Big-O(빅-오) → 상한 점근(최악의 경우)
 - 최악의 경우 : 최악의 입력 상황에서 작업을 완료하는 데 가장 연산 횟수가 많은 경우
- Big-Ω(빅-오메가) → 하한 점근(최선의 경우)
 - 최선의 경우 : 최적의 입력 상황에서 작업을 완료하는 데 가장 연산 횟수가 적은 경우
- Big-θ(빅-세타) → 위 둘의 평균



여러 가지 복잡도 표기 방법

- 입력된 아래 배열들에 대해 오름차순 정렬을 한다면?

10	20	30	40
----	----	----	----

40	30	20	10
----	----	----	----

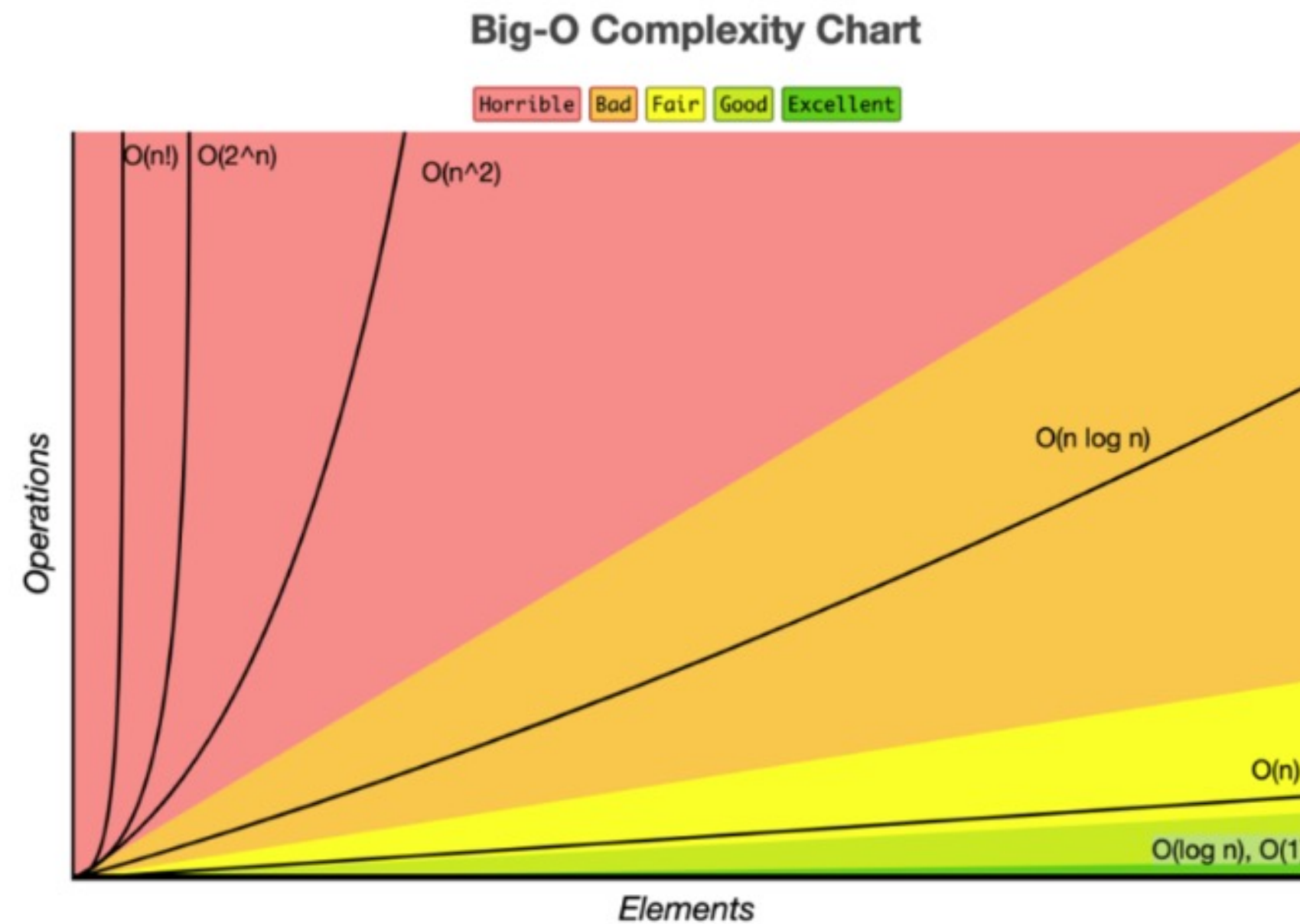
10	20	40	30
----	----	----	----

- N 의 범위 중에서 가장 큰 값이 입력되는, 최악의 경우(test case)까지 고려해야 하므로,
주로 빅-오 표기법을 사용

대표적인 Big-O 표기법의 종류

구분	$O(1)$	$O(\log N)$	$O(N)$	$O(N \log N)$	$O(N^2)$	$O(2^N)$	$O(N!)$
$N = 1$	1	1	1	1	1	1	1
$N = 4$	1	2	4	8	16	16	24
$N = 8$	1	3	8	24	64	256	40320
명칭	상수 시간	로그 시간	선형 시간	로그 선형 시간	이차 시간	지수 시간	팩토리얼 시간
예시	인덱스에 해당하는 배열의 요소 추출	이진 탐색 (이분 탐색)	for 반복문	퀵 정렬의 평균 시간 복잡도	2중 for문	피보나치 수열	순열

대표적인 Big-O 표기법의 종류



Elements : 입력값(n)의 크기
Operations : 실행 시간

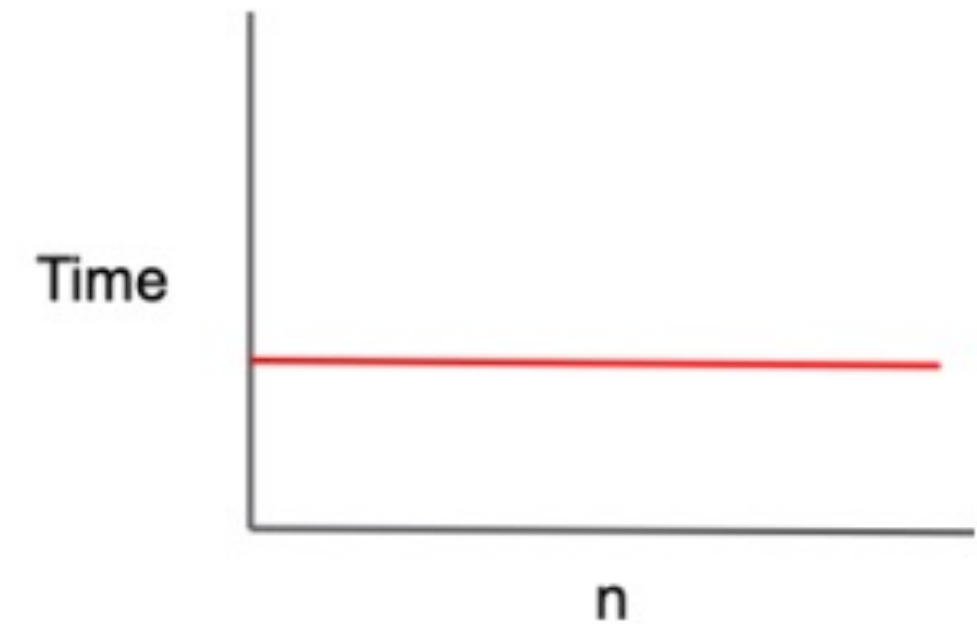
대표적인 빅오 표기법의 종류 7가지



시간 복잡도

$O(1)$

- 일정한 복잡도(constant complexity)
- 입력값이 증가하더라도 실행 시간이 늘어나지 않는다.
→ 입력값의 크기와 관계없이, 즉시 출력값을 얻어낼 수 있음
- 예 : 어떠한 배열의 특정 인덱스 위치의 요소 출력

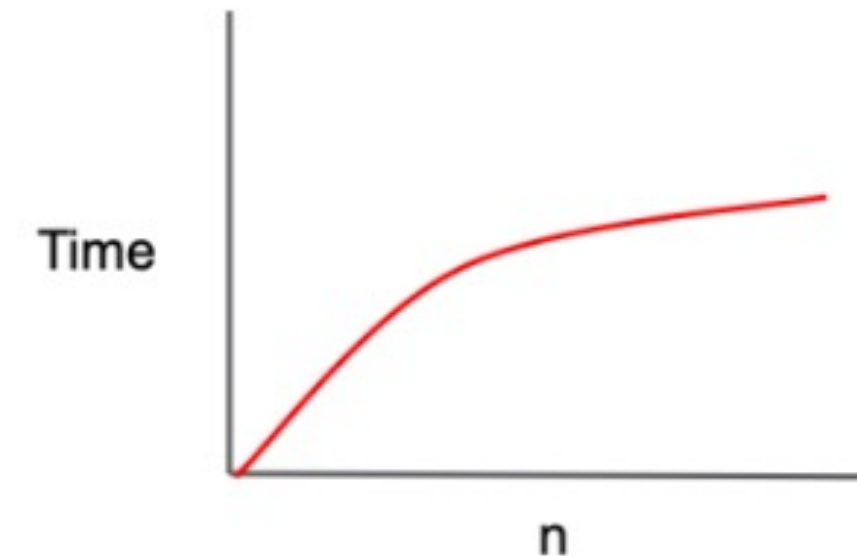




시간 복잡도

$O(\log N)$

- 로그 복잡도(logarithmic complexity)
- 시간 복잡도는 $O(1) < O(\log N) < O(N)$ 순으로 큼
 - N 이 커질수록 $O(N)$ 보다 $O(1)$ 에 더 가까워짐



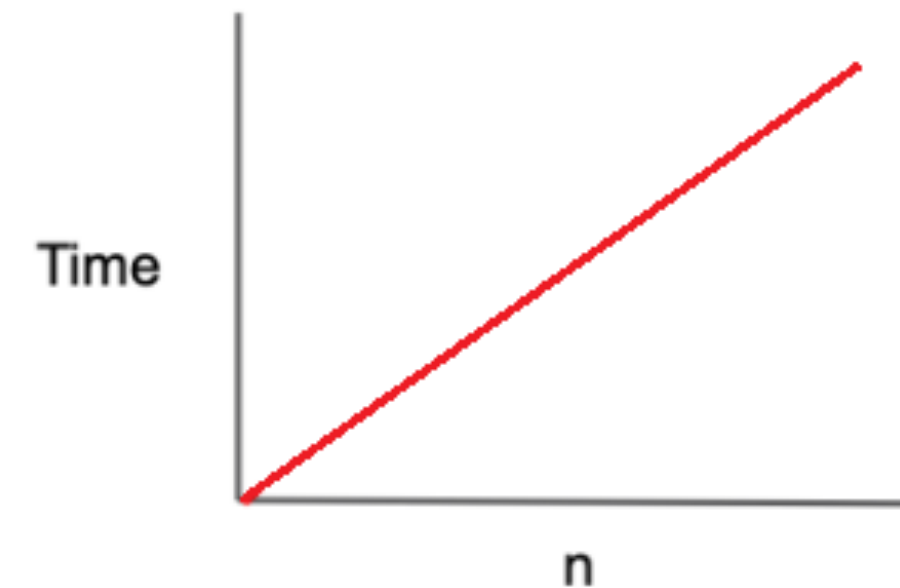
특정 숫자를 맞추는 Up & Down 게임

매번 탐색 범위의 중간값을 제시할 때마다
경우의 수가 절반씩 줄어듦



$O(N)$

- 선형 복잡도(linear complexity)
- 입력값이 증가함에 따라 실행 시간 또한 **같은 비율**로 증가
- 계수는 생략
 - 입력값(N)이 커짐에 따라 **계수의 의미가 퇴색**되기 때문
 - 아래 두 함수는 for 문의 반복 횟수가 다르지만, 모두 $O(N)$ 으로 표기($O(2N) \rightarrow O(N)$)



```
def function1(n):  
    for i in range(n):  
        print("Elice")
```

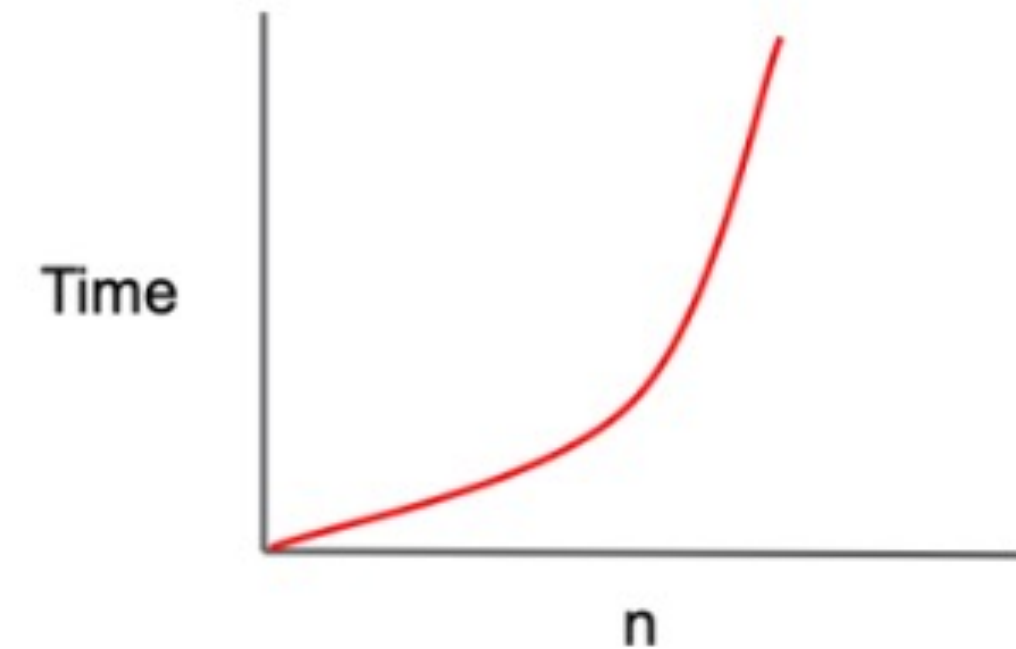
```
def function2(n):  
    for i in range(2 * n):  
        print("Elice")
```




시간 복잡도

$O(N^2)$

- 2차 복잡도(quadratic complexity)
- 입력값이 증가함에 따라 실행 시간이 N 의 제곱수의 비율로 증가
- 2중 for 문법에서 보이는 시간 복잡도 $\rightarrow O(n^2)$



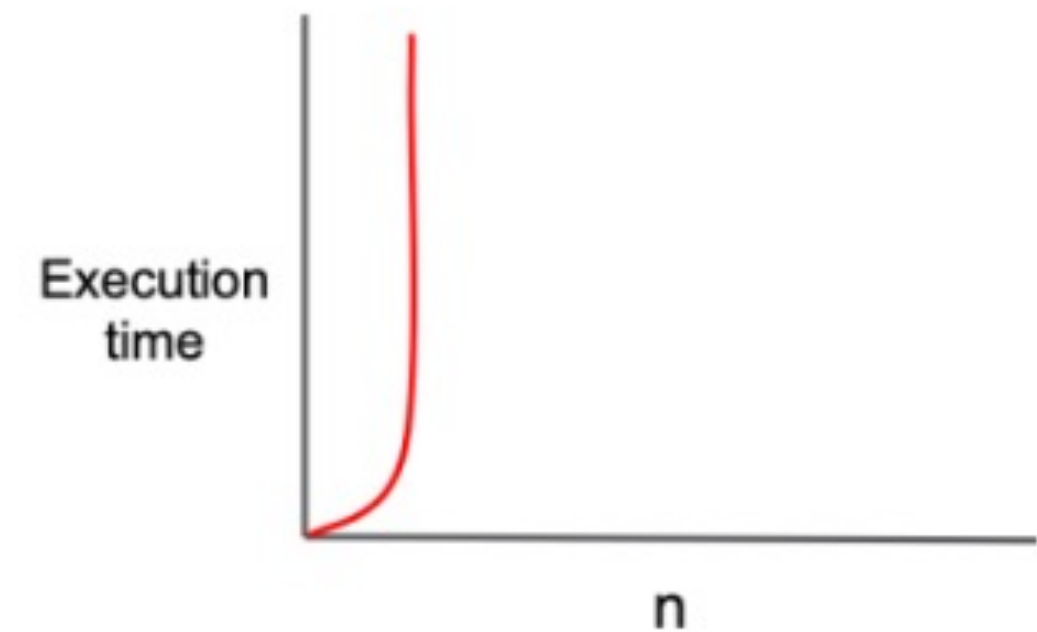
```
def function1(n):  
    for i in range(n):  
        for j in range(n):  
            print("Elice")
```

2중 for 문법



$O(2^N)$

- 지수 복잡도 or 기하급수적 복잡도(exponential complexity)
- 구현한 알고리즘의 시간 복잡도가 $O(2^N)$ 라면, 보통 다른 접근 방식을 고민해 보는 것이 좋음

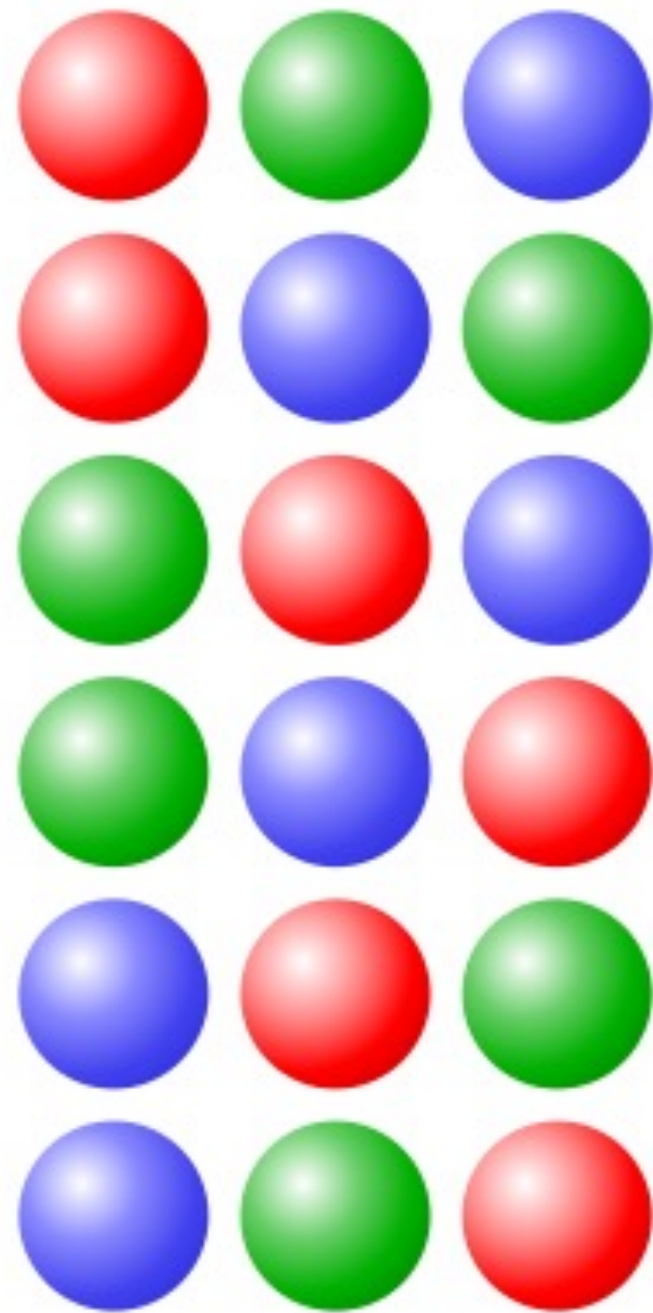


```
def fibonacci(n):  
    if n <= 1:  
        return 1  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

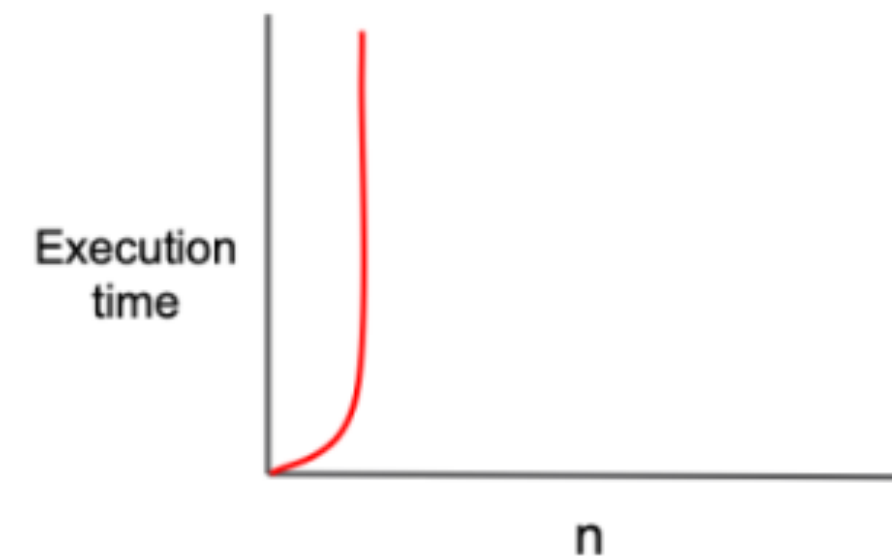
재귀를 사용한 피보나치 수열



$O(N!)$



- 팩토리얼 시간 복잡도(factorial complexity)
- 시간 복잡도가 가장 큼



3개의 서로 다른 공에 대한 총 6(3!) 가지의 순열(permutation)



예제 문제

- 크기가 N 인 배열이 있고, 배열 안의 요소들은 모두 양의 정수이며, 오름차순으로 정렬되어 있다. 이때, 배열 안에서 중복되지 않게 두 개를 골라 곱하였을 때의 최대값을 구하여라.
- 두 수를 선택하는 경우의 수는 $\frac{N \times (N - 1)}{2}$
 - 시간 복잡도: $O(N^2)$
 - N 의 값이 백만, 천만 이상의 값이 되어버리면?
- 배열의 가장 뒤에 있는 요소와 2번째로 뒤에 있는 요소를 곱함
 - 시간 복잡도: $O(1)$
 - N 의 값이 커지더라도, 보다 빠른 시간 안에 연산 가능



대략적인 시간 계산법

- Cpp가 Python과 Java보다 빠르다.
- 보통 언어별로 시간 보정이 존재하므로, 평균하여 대략적으로 1초에 1억 번 연산





Big-O 시간 복잡도 계산법칙 1

For / while loop가 한 번 중첩될 때마다 $O(N)$

```
for num in nums:
```

```
for i in range(len(nums)):
    for j in range(len(nums)):
```

```
for i in range(len(nums)):
    for j in range(len(nums)):
        for k in range(len(nums)):
```




Big-O 시간 복잡도 계산법칙 2

자료구조 사용, 다른 함수 호출에는 각각의 $O(N)$ 을 파악

```
nums = [2, 8, 19, 37, 4, 5]  
if num in nums:
```

```
nums = {2, 8, 19, 37, 4, 5}  
if num in nums:
```

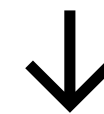
```
nums.sort()
```



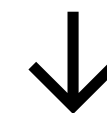
Big-O 시간 복잡도 계산법칙 3

매번 절반씩 입력값이 줄어들면 $O(\log N)$

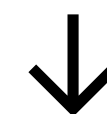
1	2	3	5	8	6	0	4
---	---	---	---	---	---	---	---



8	6	0	4
---	---	---	---



8	6
---	---



6

이진 탐색

$N = 8$

실행 횟수 = $\log(8) = 3$



실제 문항에 적용한다면?

시간 복잡도를 고민하지 않고, 무작정 구현부터 한다면
처음부터 다시 코드를 작성해야 하는 불상사가 생길 수 있음!

실제 코딩 테스트 응시 시간 내에 좀 더 많은 문항을 풀기 위해서는
이러한 연습을 많이 하는 것이 중요





실제 문항에 적용한다면?

시간 제한: 1 초
 $1 \leq N \leq 10^5$

N 이 100000까지 입력되므로 $O(N^2)$ 은 불가능

$O(N \log N)$ 의 시간 복잡도까지는 가능

이러한 방식으로, N 의 입력 범위를 통해 출제자가 의도한 풀이의 시간 복잡도를 추측할 수 있음



알고리즘에 사용되는
메모리 공간의 총량



Big-O 공간 복잡도 =

$$a = 1$$



Big-O 공간 복잡도 =

```
a = [num for num in nums]
```



공간 복잡도 계산하기

Big-O 공간 복잡도 =

```
a = [num for num in nums] for num in nums]
```



1. 문제를 정확히 이해한다
2. 문제를 해결하는 알고리즘을 개발한다
3. 알고리즘이 문제를 해결한다는 것을 증명한다
4. 알고리즘이 제한시간 내에 동작한다는 것을 보인다
5. 알고리즘을 코드로 작성한다
6. 결과를 확인한다

배열과 해시



가장 기본적인 자료 구조

```
nums = [1, 2, 3, 4, 5, 6]
```

배열의 공간 복잡도 = $O(N)$



배열 : Big-O 시간 복잡도

인덱스를 알 때 : $O(1)$

```
nums = [1, 2, 3, 4, 5, 6]
```

```
nums[2]
```



배열 : Big-O 시간 복잡도

인덱스를 모를 때 = 하나씩 검사 : $O(N)$

```
nums = [1, 2, 3, 4, 5, 6]
```

```
if 5 in nums:
```



배열 : Big-O 시간 복잡도

배열 전부 순회하기 : $O(N)$

```
nums = [1, 2, 3, 4, 5, 6]
```

```
for num in nums:
```




자료 끝에 추가하기 : $O(1)$

```
nums = [1, 2, 3, 4, 5, 6]
```

```
nums.append(7)
```



자료 중간에 추가하기 : $O(N)$

```
nums = [1, 2, 3, 4, 5, 6]
```

```
nums.insert(3, 9)
```




```
nums[2]
```

```
nums[2:5]
```

```
nums[len(nums)-1]
```

```
nums[-1]
```



배열의 한 종류, 문자들의 배열

```
tempString = "abcdef"  
for ch in tempString:
```



2차원 배열

```
nums = [[1, 2, 3, 4, 5],  
        [6, 7, 8, 9, 10],  
        [11, 12, 13, 14, 15],  
        [16, 17, 18, 19, 20]]
```



Dictionary.Key + Value (in Python)

Key는 중복될 수 없음

공간 복잡도는 대략 $O(N)$

```
studentIds = {  
    "엘리스":123,  
    "홍길동":145,  
    "김철수":563 }
```

"엘리스"	123
"홍길동"	145
"김철수"	563
...	...



Key를 이용해서 Value 가져오기 : 대략 $O(1)$

```
print(studentIds["엘리스"])
```



Key가 존재하는지 확인하기 : 대략 $O(1)$

```
if("엘리스" in studentIds):  
if("이영희" in studnetIds):
```




Key, Value 추가하기 : 대략 $O(1)$

```
studentIds["이영희"] = 938
```



해당 Key의 Value 변경하기 : 대략 $O(1)$

```
studentIds["엘리스"] = 555
```



해시의 공간 복잡도 = $O(N)$

해시는 데이터가 입력되지 않은
여유 공간이 많아야 성능 유지

Value 없이 Key만 있는 Dictionary

```
studentNames = {"엘리스", "홍길동", "김철수", "이영희"}
```

"엘리스"
"홍길동"
"김철수"
"이영희"



해시

식별자가 있는 데이터, 시간 복잡도 ↓ / 공간 복잡도 ↑

배열

식별자가 없는 데이터, 시간 복잡도 ↑ / 공간 복잡도 ↓